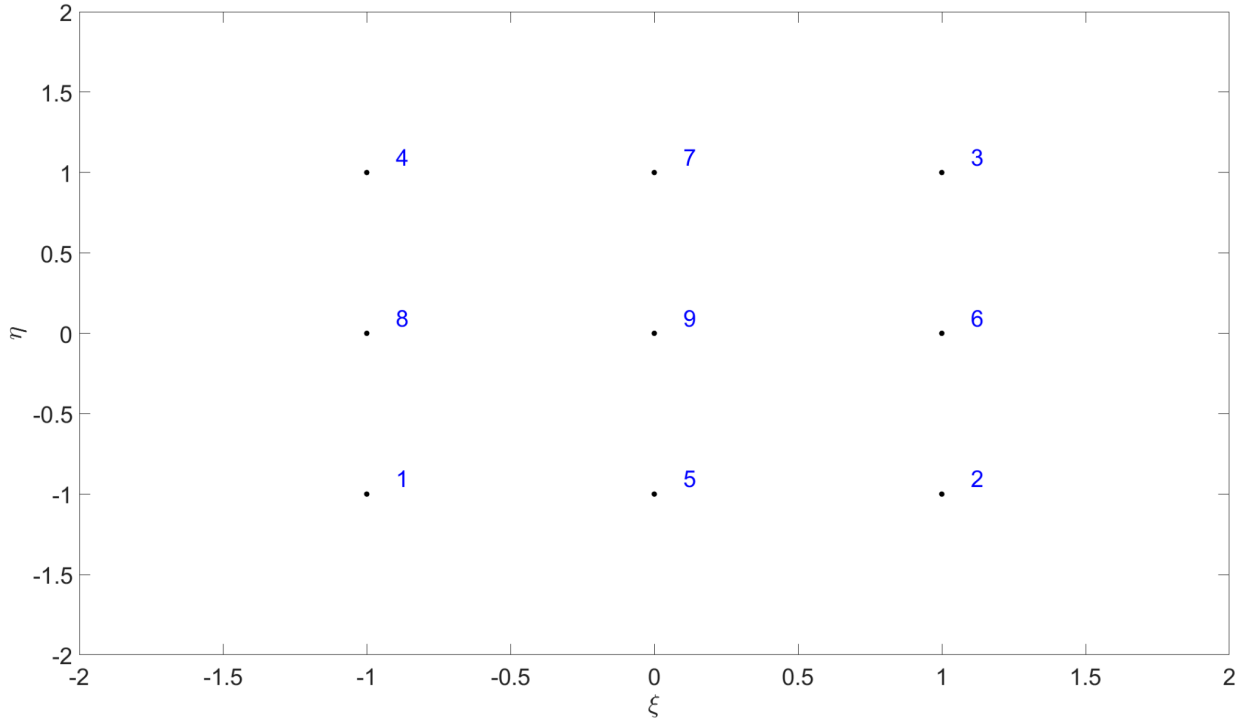
FEA HW4

Ben Sarfati 885573816

Problem 1.1



This is a biquadratic element in reference configuration with local node numbering in blue.

Problem 1.2, 1.3

Code attached (+Bonus).

Problem 2.1

After initializing the node coordinates list in global configuration, the isoparametric mapping is assigned according to the basis functions provided in the tutorials for both biquadratic and quadratic triangular elements:

```
type = 'biquadratic';

%Define isoparametric mapping
phi = @(xi,eta) (1/4)*[
xi*(xi-1)*eta*(eta-1)
xi*(xi+1)*eta*(eta-1)
xi*(xi+1)*eta*(eta+1)
xi*(xi-1)*eta*(eta+1)
2*(1-xi^2)*eta*(eta-1)
2*xi*(xi+1)*(1-eta^2)
2*(1-xi^2)*eta*(eta+1)
2*xi*(xi-1)*(1-eta^2)
4*(1-xi^2)*(1-eta^2)]';
```

```
type = 'quadratic triangular';

%Define isoparametric mapping
phi = @(xi,eta) [
xi*(2*xi-1)
eta*(2*eta-1)
(1-xi-eta)*(1-2*xi-2*eta)
4*xi*eta
4*eta*(1-xi-eta)
4*xi*(1-xi-eta)]';
```

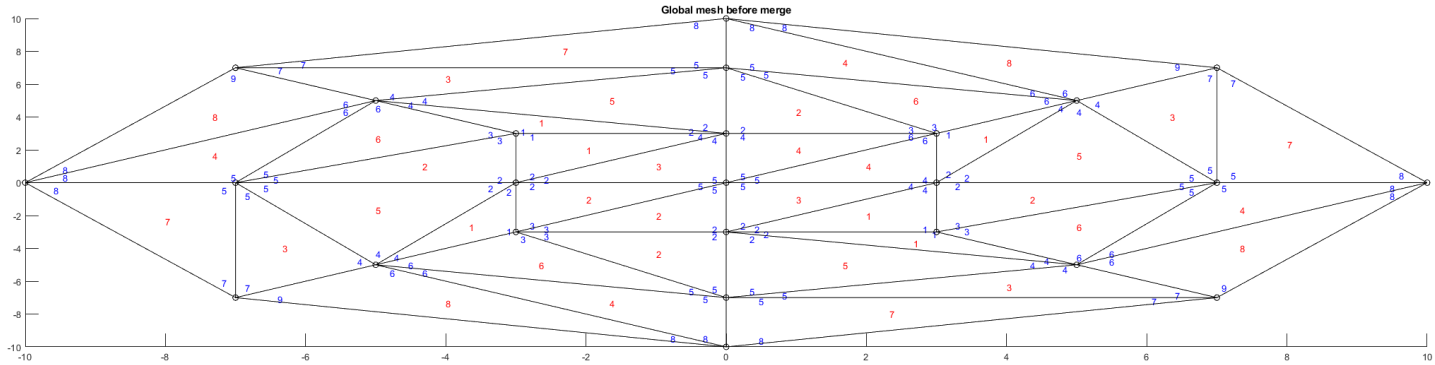
These can be found in the attached *meshElement* function.

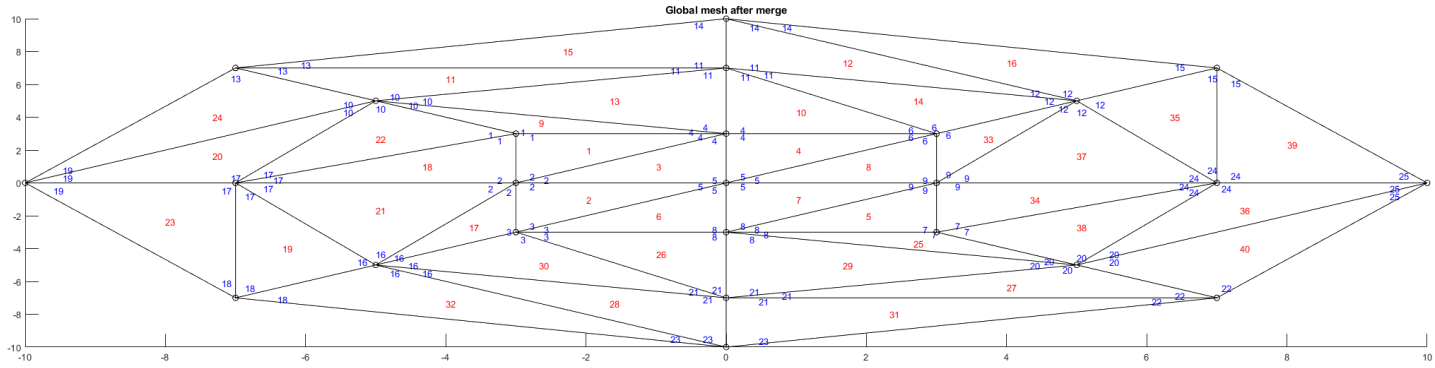
Problem 2.2

Code attached.

Problem 2.3

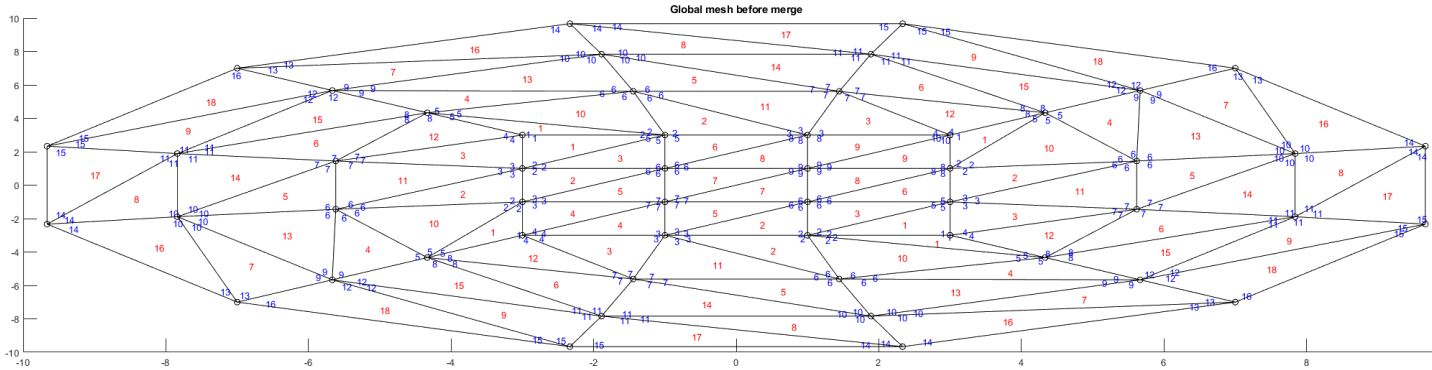
After initializing the node coordinates list and element list as described in problem 2's header, each macro element is meshed using the written *meshElement* function with a mesh refinement factor that is consistent across all elements. Afterward the elements are meshed together using mergeMesh which is provided, and everything is plotted exactly as in the example code snippet. Below is the result of a mesh refinement factor of 2:

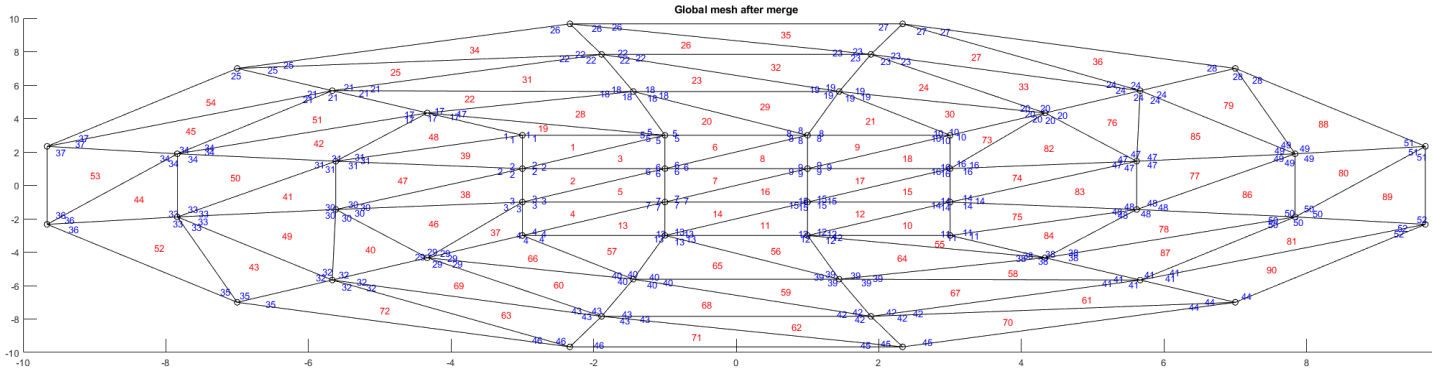




Next page vvvvvvvvvvv

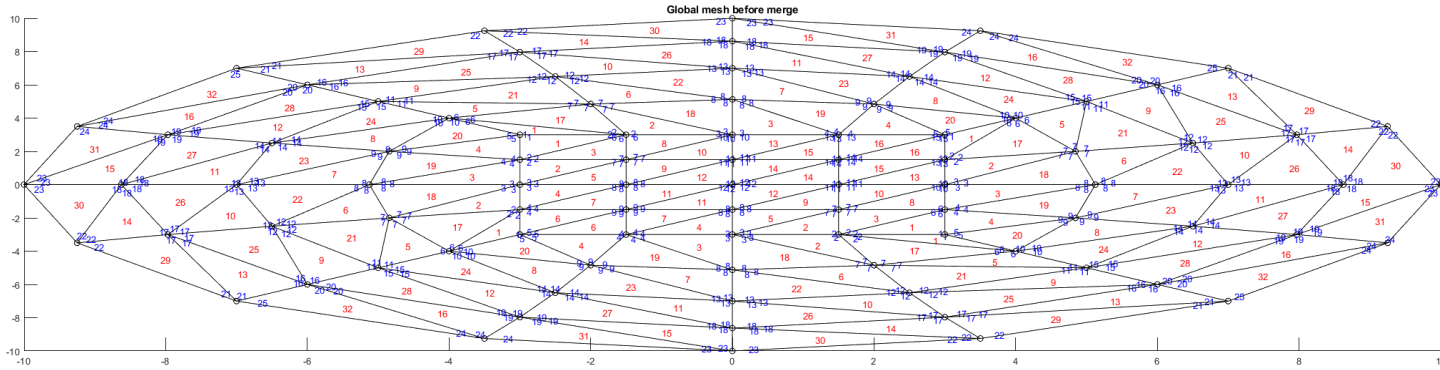
A mesh refinement factor of 3:

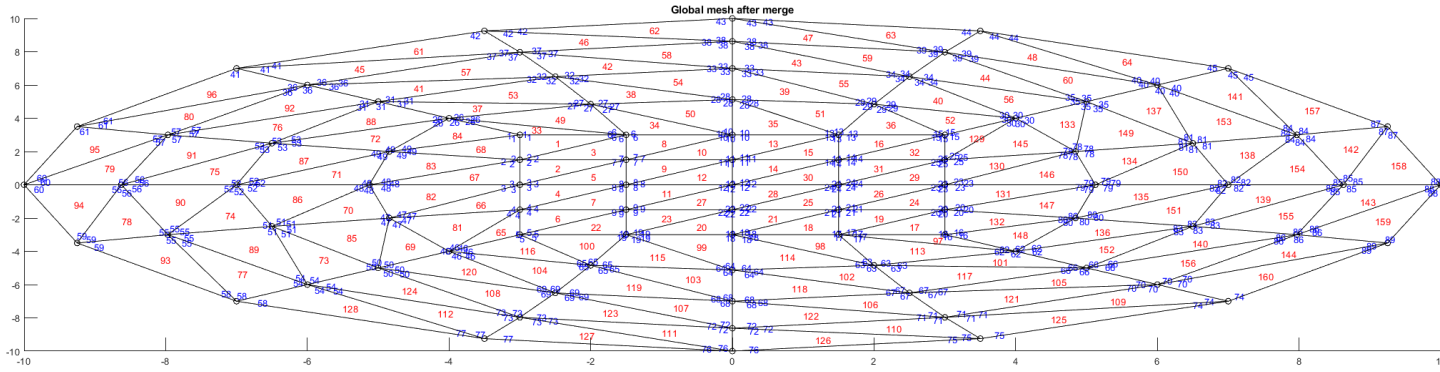




Next page vvvvvvvvvvv

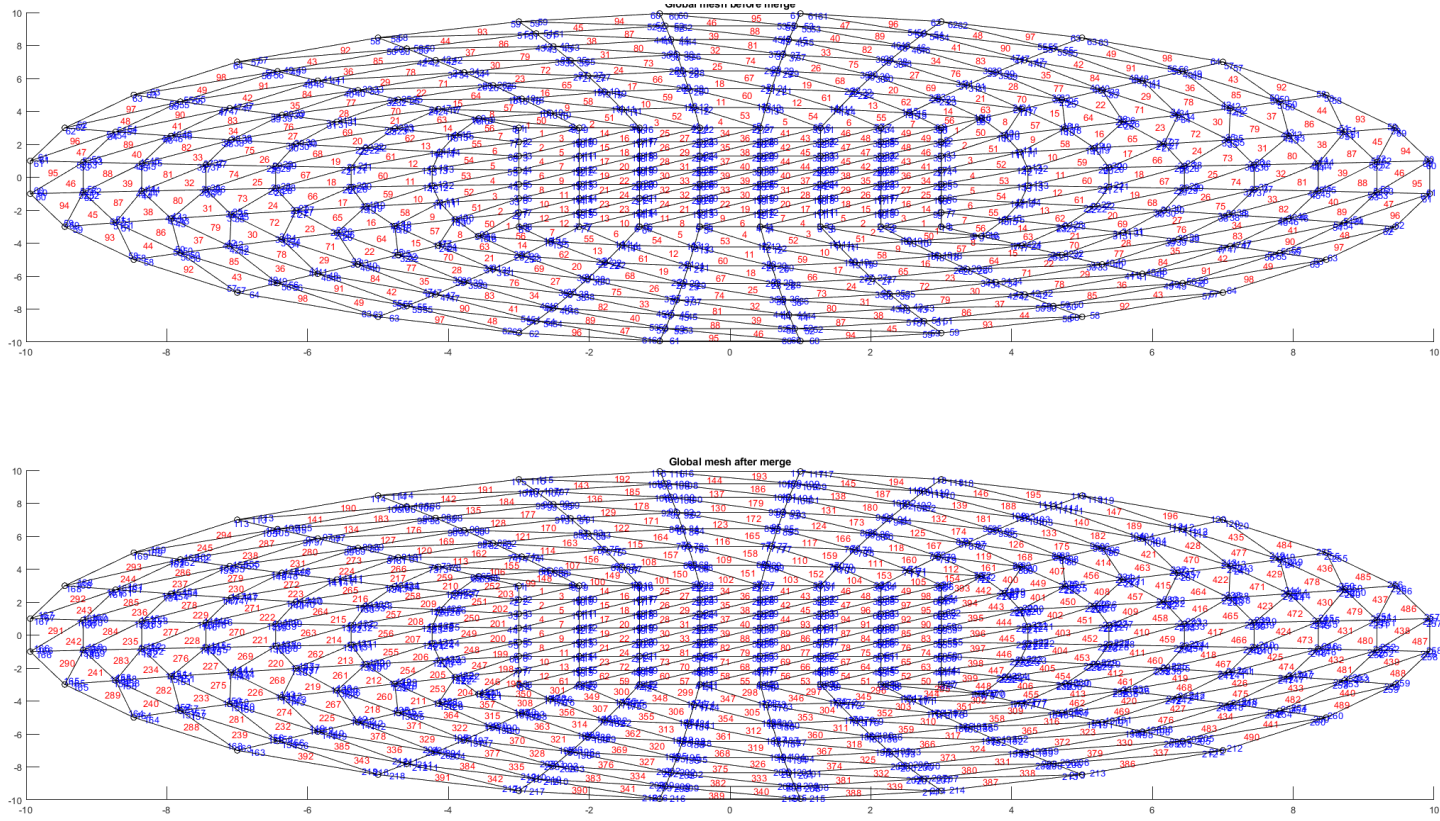
A mesh refinement factor of 4:





Next page vvvvvvvvvvv

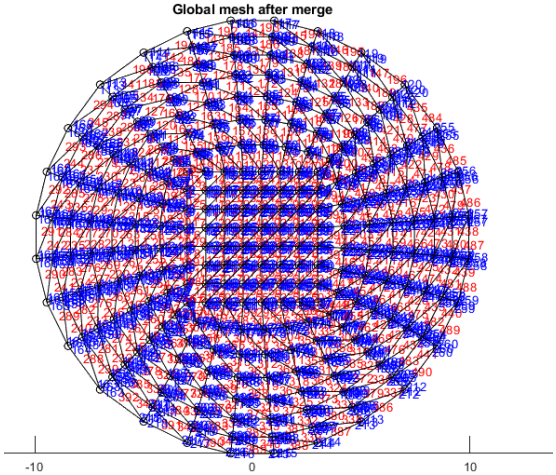
And a mesh refinement factor of 7:



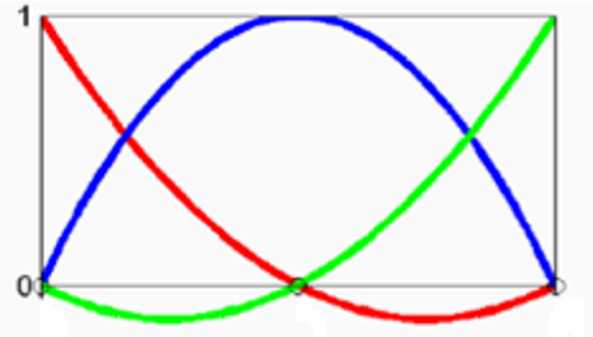
The provided script FEA_HW3 can also generate these plots on demand provided it has access to *meshElement*, *innerMesh*, and *mergeMesh*.

Problem 2.4

The finer meshes visibly result in a rounder shape. The aspect ratio above makes it seem like an oval shape but considering the axes it's closer to a circle shape. In fact, if we write *axis equal* in the command window we get



Of course this isn't a real circle; it's just a polygon with vertices which lie more or less on a circle. As the number of these vertices increases the polygon looks more and more like a circle. But is it really converging to a real circle?
The answer should be no because intuitively this would only happen if all outer nodes lied on a true circle. the distance of the boundary midline nodes to the origin is 10, whereas the distance of the corner nodes on the boundary to the origin is the square root of 2*7^2 which is sqrt(98), which is just less than 10. So this is probably not converging to a true circle. So what is going on?
This is interesting because nothing directly "told" the outer nodes to create this shape. In order to understand this we have to look at the isoparametric mapping of boundary nodes. These boundary nodes, which have an eta value of 1 in reference, map to the global configuration as a linear combination of the 3 top global node coordinates with the corresponding basis functions evaluated at the corresponding reference coordinates. These basis functions are quadratic; on the line eta=1 they probably look like the quadratic line element basis functions:



$\xi_1 = -1$ $\xi_2 = 0$ $\xi_3 = 1$

So, when mapping these boundary nodes, it is logical that they end up not on a line between the element's boundary node and the next, but instead on a unique parabola formed from one edge node to the next, which intersects the midline node. Again, as mentioned, the fact that these nodes are almost equidistant from the origin creates a quasi-circular shape.